Action-Grounded Symbols: Abstracting Symbols from Skills in a Mobile Robot

Klara Skorupowska

2026

Abstract

Key words: artificial intelligence, abstract symbols, computational cognitive science

Contents

1 Introduction **4**

2 Theoretical Foundations **5**
2.1 Embodied Cognition 5
2.2 Symbol Grounding Problem and Symbol Emergence 6
2.3 Sensory Input: Visual Processing 6
2.4 Symbol Representation: Growing Neural Gas 7

3 High-Level Idea **9**
3.1 Starting Point 9
3.2 Algorithm Description 9
3.3 Abstracting Symbols 9

4 Implementation **10**
4.1 Environment 10
4.2 Supervisor 12
4.3 Agent 12

5 Results **15**

6 Discussion **18**

1 Introduction

To be replaced, when body is ready

Imagine an embodied agent - a robot. We want it to complete a task for us. It has a library of ready-to-use skills. To complete the task, the robot needs to execute those skills in correct order. But, for slightly different task, or in different circumstances, some skills may be unnecessary, or the order needs to be change. We want our robot to smoothly adapt to the changes in an environment and objectives, and only focus on what is important for specific skill.

Take a closer look at the skills. A skill is a list of low-level actions, that can be executed by this robot. Their execution ends either successfully, or fail at some action. Obviously, the robot should execute a skill only when it can assume that the execution is going to be successful. For that each one comes with a precondition. A precondition is a symbol that says "You can do this skill now". This symbol is a predicate resolvable by examining the perceptual state of the robot. Robot can take a look around and decide if a skill is save to execute. Then, after skill execution, the robot changes its perceptual state, either by changing something in the environment, or changing its location. This new state is another predicate: an effect. Since the robot is supposed to execute skills successfully, only effects of those are of any interest to it. Then robot uses those predicates to establish which skills needed to be done, and in which order to complete assigned task. The aim of this project is to let the robot learn those predicates on its own.

This project is a replication of the work of Konidaris et al. (2018). The project's high-level idea and philosophical problem at a basis are identical to those of Konidaris'. I introduced some minor changes to the implementation in order to facilitate replication and, at crucial aspects, get closer to biological accuracy. As a result... The conclusion is...

Paper structure

Chapter 2 of this paper presents theoretical grounds, used tools, and assumptions behind this project.

In chapter 3, the algorithm is described as a high-level idea, with no implement-specific details. Whereas, implementation is shown in chapter 4.

Chapter 5 shows obtained symbols, and results for completing the task, as well as learning statistics.

Chapter 6 discusses those results, and indicates future steps for this project.

2 Theoretical Foundations

2.1 Embodied Cognition

According to the movement of embodied cognition, embodiment is inseparable part of the cognition (Varela et al., 2017). This stands in opposition to more classical cognitive science, representing computational theory of mind, where brain is a processor that handles abstract information (Pylyshyn, 1984).

To formalise the term of embodied cognition, Wilson (2002) introduced the six views. It is six statements that define what it means for the cognition to be embodied. In the context of this project the most interesting one is the fifth, although rest of them are also fulfilled.

Fifth view: Cognition is for action. This means that the mind's function is to guide action. Every mechanism, process or representation within the mind has to come from the need of planning or executing actions. Mind's main task is to control the movement of the body in an environment and ensure its survival.

The movement of embodied cognition is founded, among others, on the theory of affordances by Gibson (1977). His theory states that we perceive the world in terms of what can be done, how can we interact with an environment, or, as used in this project, what skill can be executed. For example, perceiving a door is seeing them as an object that can be opened and passed through.

Embodied cognition is criticized. Radical variants of embodied cognition face significant theoretical challenges. For example, by the poverty of stimulus argument, which states that there is not enough information and structure in environment to solve cognitive problems (Miłkowski, 2017). Miłkowski settles on a hypothesis that for cognition both realms are important, embodiment and computations. Embodied experimental designs do lead to interesting discoveries, but not all cognitive processes can be explained by it.

revisit this

This project focuses on what embodiment can do. Specifically, the agent learns to perceive its environment through action-oriented predicates. Those predicates closely resemble Gibson's affordances and are learn by continuous interaction with an environment.

2.2 Symbol Grounding Problem and Symbol Emergence

Symbol Grounding Problem (SGP) was introduced by Harnad (1990). His starting point is computational theory of mind that looks at a brain as open-loop computing machine that operates within symbol systems. He drew attention to the meanings of those symbols, arguing that any interpretation for them, must be extrinsic to the system. He compares understanding such symbols to learning Chinese solely from the Chinese dictionary. Every attempt to find a meaning leads only to more symbols, that still lack the meaning. The Grounding Problem revolve around this. It asks the question if there is a mechanism that allows finding the meanings of symbols without escaping the system, therefore making them intrinsic to it. This is one side of the problem, in a context of robotics and cognitive modelling, the essential challenge is not symbol grounding. The most interesting problem there is symbol emergence: not only how to find the meaning of existing symbols, but how meaningful symbols arise, evolve, and become organized through social and embodied interaction (Taniguchi et al., 2016).

Harnad himself proposed a solution to SGP (Harnad, 1990). The mind grounds symbols in environment by creating iconic and categorical representations. It is a mechanism of associating the sensations with objects causing them. An object is then described in terms of sensory projection: as a representative point in perceptual space (an icon) or multiple points' invariants (a category). Even though, Harnad do not point out what triggers this mechanism, Peirce's pragmatism effectively addresses this issue. It ties objects and their characteristics with actions they are associated with (Kato, 2026). For example, a door is described as something that can be opened.

This and other solutions, from different angles, were proposed, yet the problem remain opened. Taddeo and Floridi (2005) conducted a review of 15 years of solving the SGP. They concluded that explanations rely on pre-installed or other way given semantic resources.

This project likewise depends on external structures and symbol manipulation. The symbols essential for operating within an environment are obtained by repetitive discovery of possible sensory projections and binding them to the actions, they are not pre-installed but learned through interaction with an environment. The symbols are then used for planning and executing a task using planning algorithms.

2.3 Sensory Input: Visual Processing

Biological sensory input is processed through neural paths before it reaches the brain. Even there, this early processed input can be further manipulated. (Crick & Koch, 2003). The robot under consideration has two types of sensors: LIDARs and RGB camera. Since humans do not possess anything remotely like the LIDARs, I focus on the vision.

The subject of recognising colours is described by Livingstone and Hubel (1988). They described how the eyes and the brain analyse visual input. Colour is perceived by the cells in the retina called cones. There are three types of cones: red-, green- and blue-sensitive. This is enough to distinguish between colours.

Another type of processing that happens at retinal level is detecting the differences in light saturation. They allow us to see the edges of objects in our visual field. Those cells are called retinal ganglion cells, and they form receptive fields. A receptive field has two areas: the on-centre and the off-centre. If the on-centre cells perceive more photons than the off-centre ones, the on-centre is activated. If the off-centre perceives more, the off-centre is activated. If both perceive the same amount, there is no activation (Kuffler, 1953).

Kuffler assumed some things that were proven wrong. First example is the linearity of the ganglion cell receptive fields. Enroth-Cugell and Robson (1966) proved that this is an oversimplification. Complete linearity means that the linear change in lighting is translated to the linear reaction of the cells in full input range. Yet some receptive fields are more sensitive to darkness, some to brightness.

Another example of known flaw in Kuffler's model is the laboratory setting of creating it. Keffler used tiny spots of light on anesthetised cat's retina. Yet in natural condition, the lighting changes, the eye moves. Later it was proven that the receptive fields adapt dynamically to those changing conditions (Smirnakis et al., 1997).

Even though Keffler's model is a huge oversimplification, it is still a basis for the next models. I decided to use this simple model in this project, since it is a proof of concept.

For this project, the visual information from RGB camera matrix is translated to activation of red, blue, and green cones, and activation of on-centre and off-centre fields. Such pre-processed visual information is passed further to the symbol emergence algorithm.

2.4 Symbol Representation: Growing Neural Gas

Symbolic representations describe raw sensory input in a more abstract way enabling reasoning, planning, and acting. In the philosophy of mind, the ontological status of representations is a subject of debate. Fodor (1975) proposed that the mind is a symbolic system, and that mental representations are symbolic. Smolensky (1988) argued that those representations are not symbolic, but rather subsymbolic. In this connectionists view, representations are not discrete symbols, but rather continuous patterns of activation in neural networks. Radical anti-representationalists and enactive paradigms argue that the thoughts are neither symbolic nor subsymbolic, but rather non-representational. They are patterns of sensorimotor activity (Brooks, 1991).

This work assumes that the representations are subsymbolic. The representations can be presented as a topological map of the perceptual space and manipulated in a symbolic manner.

A well-established algorithm for creating topological maps is the Self-Organizing Map (SOM) algorithm.(Kohonen, 1982; Ritter & Kohonen, 1989). It assumes fixed structure of the map, such as a grid. Then it is trained to represent the input space by adjusting positions of the existing nodes. The Growing Neural Gas (GNG) algorithm (Fritzke, 1994) extends the SOM framework. It does not assume a predefined structure. It starts with two nodes, grows by adding new ones in areas of high error, and removes ones in unvisited areas. This allows the GNG to adapt to the input space and represent it more accurately.

In this project, the GNG algorithm encodes sensory observations corresponding to successful executions of skills. Then, symbols relevant for the task are abstracted through the mathematical operations on these topological maps.

3 High-Level Idea

3.1 Starting Point

This project is a replication of the work of Konidaris et al. (2018). They proposed a framework for abstracting grounded symbols from predefined skills. Their framework collected data from repetitions of skills in a form of 3D representation as cloud of points. Then, it uses clustering methods, such as DB-SCAN to find what the skill changes in the data. In the learning setting, the robot had two arms, mobile platform, and a binocular vision. The task setting was a room with a bottle in a closed fridge and a closed cupboard. The robot was supposed to move a bottle from the fridge to the cupboard. The skills, such as opening the fridge, were repeatedly executed, starting from a known initial state. The learning data consisted of initial states and final states of successfully completed trials. Then they used the clustering algorithm to find clusters of repetitive data through all the repetitions. This way they obtained symbols that could be interpreted as "Fridge is open", or "robot is holding the bottle".

3.2 Algorithm Description

Fist Loops

Successful Operating

Case Analysis

3.3 Abstracting Symbols

big change - need to change a lot of this

4 Implementation

Algorithm is implemented in Python using PyBullet simulation. The program keeps the simulation and the logic completely separated. This way the simulation keeps the realism of operating within the real environment. Thus the program consists of three main part: simulation, agent and supervisor. Each module is run in separate thread, mimicking the ROS nodes. Modules communicate through the bus, using services and broadcasting.
Simulation module generate the environment and control physics. Supervisor module has the knowledge about the simulation an the robot. It knows absolute position of the robot in the room as well as currently executed skill.
Agent module has implementations of operating logic, abstracting symbols, updating predicates, and low-level skills.
This chapter provides an overview of the system architecture and the methods employed in the implementation. The full source code repository is available online (Skorupowska, 2026).

4.1 Environment

A robot is a simplified version of e-Puck 2 (Figure 1). It resembles its shape and have the same set of sensors and actuators. The only actuators are two differential wheels, which makes the robot to move non-holonomically. The sensors are: 8 LIDARs with range from 0 to 10 cm and a RGB camera with matrix of 120×160 pixels.

The arena (Figure 2) for the task consist of two rooms separated by the wall. Each room has different coloured floor so the robot is able to different between them using only its sensors. In the separating wall there is a contrasting door. The door swings open on the push of the robot. Robot's task originate in orange room, then in second room, the blue one, there is contrasting post, which robot is supposed to reach.

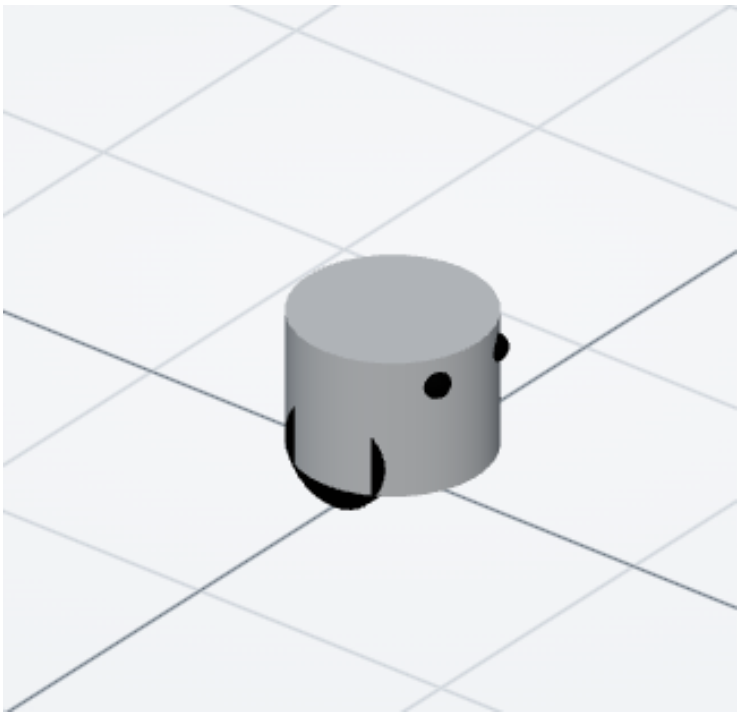


Figure 1: Simplified model of an e-Puck 2 as cylinder body with differential wheels. Eyes added purely for aesthetics.

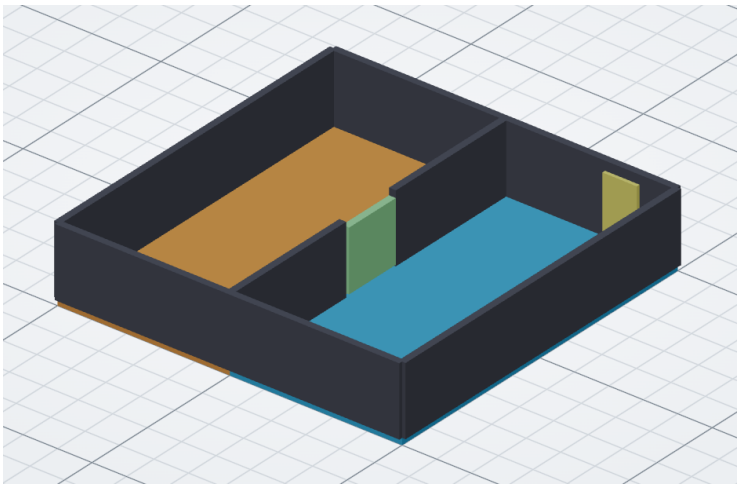


Figure 2: Model of an arena used in the simulation. Two rooms separated by a wall with the door.

4.2 Supervisor

Next module updates the data for all the debugging figures and answers robot's questions through the bus. This is the implementation of supervision within the learning. The supervisor has knowledge beyond robot's perception and can modulate both the environment and robot. The main task of a supervisor in the learning process is checking if a skill was executed successfully, Each skill has some conditions of success and failure. This is similar to what is being learned, but the difference is that those are formulated in terms of the outsider. The outsider can have different sensors or simply different point of reference. For example, the supervisor knows in which room the robot is, the robot does not. The skill 'start' can be executed anywhere, has no precondition, but is finished successfully only if the robot is in the orange room. Supervisor checks this condition and returns to the agent if it succeed.

4.3 Agent

This module consist of the main logic loop, the networks for predicates and transitions, and low-level skills.

Predicates

Predicates are represented by Supervised Growing Neural Gas. These are implementation of basic GNG Fritzke (1994) with added behaviour for negative points. The original algorithm was for unsupervised learning, and mapped space from collected points. Since the execution can be successful or not, I added a reverse logic for negative points, symmetrical to treating positive ones. If a point considered is positive, the neural gas points are pulled to it. If it is negative, they are pushed away. add parameters with explanation for them

Transition graph

The transition graph is represented by a standard directed graph. It plays no role in the learning process. This graph is used for generating PDDL files. The transitions are collected from the overlapping points of effects and preconditions and generated after each update to either of them.

PDDL files

PDDL files are generated by simple, structural code. Each time the predicates are updated, the new problem and domain files are generated. The domain file (Listing 2) defines the world for an agent. What can be seen (predicates) and what can be done (actions). In this domain there are as many actions as skills, and for each skill two types of predicates: can-run-skill and executed-skill. This way we cover both starting conditions for a skill and its effect. The problem file (Listing 1) specify the domain and the predicates that are fulfilled at the beginning and the predicates that need to be fulfilled at the end.

```
(define (problem plan_problem)
  (:domain robot_domain)
  (:init
    (can_run_Start)
  )
  (:goal
    (executed_Finish)
  )
)
```

Listing 1: Problem definition in PDDL

```
(define (domain robot_domain)
  (:requirements :strips)
  (:predicates
    (can_run_Open_the_Door)
    (executed_Open_the_Door)
    (can_run_Start)
    (executed_Start)
    (can_run_Finish)
    (executed_Finish)
    (can_run_Go_to_the_Goal)
    (executed_Go_to_the_Goal)
    (can_run_Go_to_the_Door)
    (executed_Go_to_the_Door)
    (can_run_Find_the_Door)
    (executed_Find_the_Door)
    (can_run_Find_the_Goal)
    (executed_Find_the_Goal)
  )

  (:action Open_the_Door
    :parameters ()
    :precondition (can_run_Open_the_Door)
    :effect (and (executed_Open_the_Door) (can_run_Find_the_Door) (
    can_run_Find_the_Goal) (not (can_run_Open_the_Door)))
  )

  (:action Start
    :parameters ()
    :precondition (can_run_Start)
    :effect (and (executed_Start) (can_run_Find_the_Door) (not (
    can_run_Start)))
  )

```

```
  (:action Finish
    :parameters ()
    :precondition (can_run_Finish)
    :effect (and (executed_Finish) (not (can_run_Finish)))
  )

  (:action Go_to_the_Goal
    :parameters ()
    :precondition (can_run_Go_to_the_Goal)
    :effect (and (executed_Go_to_the_Goal) (can_run_Finish) (
    can_run_Find_the_Door) (can_run_Find_the_Goal) (not (
    can_run_Go_to_the_Goal)))
  )

  (:action Go_to_the_Door
    :parameters ()
    :precondition (can_run_Go_to_the_Door)
    :effect (and (executed_Go_to_the_Door) (can_run_Open_the_Door)
    (can_run_Find_the_Door) (not (can_run_Go_to_the_Door)))
  )

  (:action Find_the_Door
    :parameters ()
    :precondition (can_run_Find_the_Door)
    :effect (and (executed_Find_the_Door) (can_run_Go_to_the_Door)
    (not (can_run_Find_the_Door)))
  )

  (:action Find_the_Goal
    :parameters ()
    :precondition (can_run_Find_the_Goal)
    :effect (and (executed_Find_the_Goal) (can_run_Go_to_the_Goal)
    (not (can_run_Find_the_Goal)))
  )
)
```

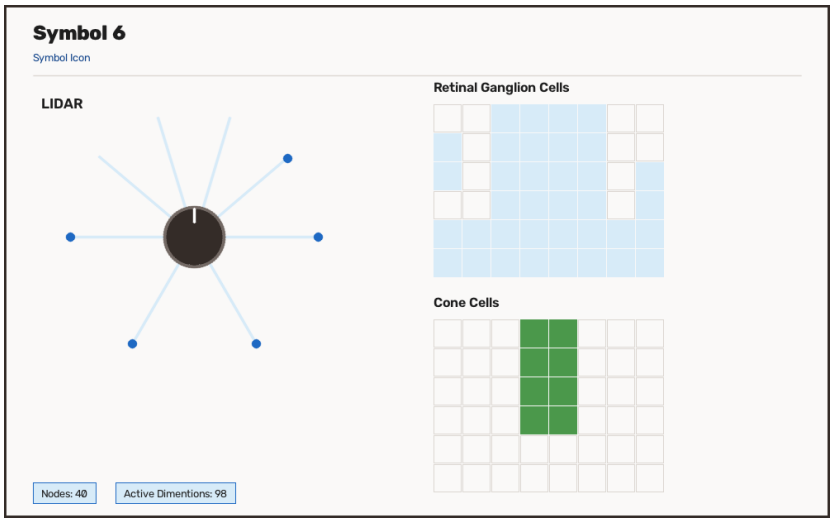
Listing 2: Domain definition in PDDL

Visual Preprocessing

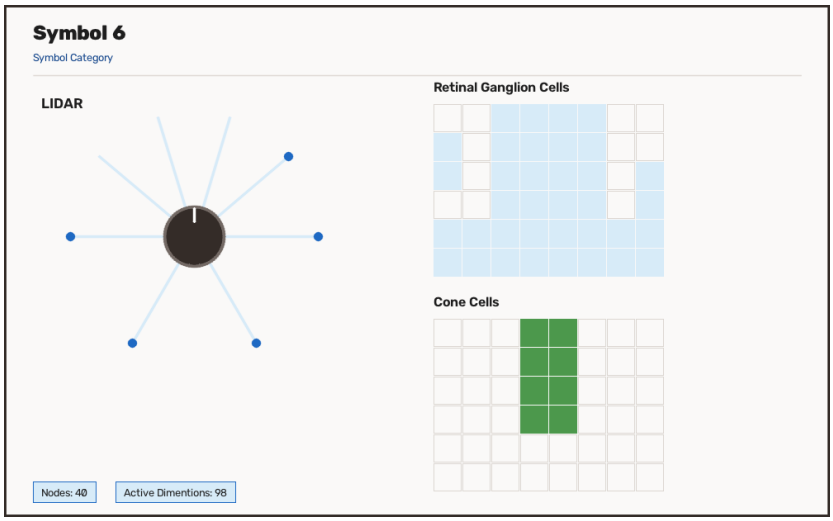
Low-level Skills

5 Results

The main results of this project are symbolic preconditions and effects. Appendix **??** holds all obtained symbols. Figure 3 shows two types of representing 'Finish' skill precondition. First type, the iconic representation (fig. 3a), shows median point of the GNG. Second type, the categorical representation (fig. 3b), shows the invariant dimensions for all the points in GNG. In this example, we see that to be able to finish successfully, the robot has to see yellow panel in front of him.



(a) Iconic representation



(b) Categorical representation

Figure 3: 'Finish' skill precondition symbolic representation. Two types of visualising obtained symbols: iconic and categorical.

Those symbols are the result of executing the main loop 32,216 times in 10 sessions. Within those executions was exploration as well as planning. The brain was updated 12,887 times. Figure 4 shows the frequency of brain updates through time. This shows that the more the agent knows the less frequent the updates are.

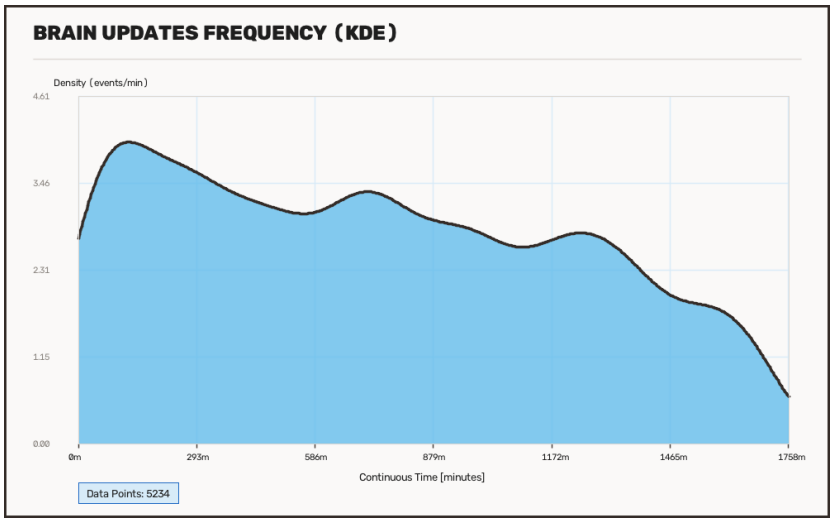


Figure 4: Update frequency through all the sessions. For calculating the update density (frequency) the Kernel Density Estimation (KDE) was used.

The symbols dictate possible transitions between skills. As mentioned before, the transition is possible if an effect of preceding skill is within the precondition of following one. Figure **??** shows the transitions deducted from the predicates. In given task the robot searched for the path from 'Start' to 'Finish'.

Tests

For testing the planning was executed with fixed goal skill set to 'Finish'. The starting skill was chosen randomly among the active ones.

6 Discussion

References

Brooks, R. A. (1991). Intelligence without representation. *Artificial intelligence*, *47*(1-3), 139–159.

Crick, F., & Koch, C. (2003). A framework for consciousness. *Nature neuroscience*, *6*(2), 119–126.

Enroth-Cugell, C., & Robson, J. G. (1966). The contrast sensitivity of retinal ganglion cells of the cat. *The Journal of physiology*, *187*(3), 517–552.

Fodor, J. A. (1975). *The language of thought* (Vol. 5). Harvard university press.

Fritzke, B. (1994). A growing neural gas network learns topologies. *Advances in neural information processing systems*, *7*.

Gibson, J. J. (1977). The theory of affordances. *Hilldale, USA*, *1*(2), 67–82.

Harnad, S. (1990). The symbol grounding problem. *Physica D: Nonlinear Phenomena*, *42*(1-3), 335–346.

Kato, T. (2026). Pragmatism: What is the meaning of symbols? In *Symbol emergence systems: An interdisciplinary discussion about cognition, language and society* (pp. 15–19). Springer.

Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological cybernetics*, *43*(1), 59–69.

Konidaris, G., Kaelbling, L. P., & Lozano-Perez, T. (2018). From skills to symbols: Learning symbolic representations for abstract high-level planning. *Journal of Artificial Intelligence Research*, *61*, 215–289.

Kuffler, S. W. (1953). Discharge patterns and functional organization of mammalian retina. *Journal of neurophysiology*, *16*(1), 37–68.

Livingstone, M., & Hubel, D. (1988). Segregation of form, color, movement, and depth: Anatomy, physiology, and perception. *Science*, *240*(4853), 740–749.

Miłkowski, M. (2017). Situatedness and embodiment of computational systems. *Entropy*, *19*(4), 162.

Pylyshyn, Z. W. (1984). *Computation and cognition: Toward a foundation for cognitive science*. The MIT Press.

Ritter, H., & Kohonen, T. (1989). Self-organizing semantic maps. *Biological cybernetics*, *61*(4), 241–254.

Skorupowska, K. (2026). *Abstract symbols: Bachelor's degree implementation*. GitHub.

Smirnakis, S. M., Berry, M. J., Warland, D. K., Bialek, W., & Meister, M. (1997). Adaptation of retinal processing to image contrast and spatial scale. *Nature*, *386*(6620), 69–73.

Smolensky, P. (1988). On the proper treatment of connectionism. *Behavioral and brain sciences*, *11*(1), 1–23.

Taddeo, M., & Floridi, L. (2005). Solving the symbol grounding problem: A critical review of fifteen years of research. *Journal of Experimental & Theoretical Artificial Intelligence*, *17*(4), 419–445.

Taniguchi, T., Nagai, T., Nakamura, T., Iwahashi, N., Ogata, T., & Asoh, H. (2016). Symbol emergence in robotics: A survey. *Advanced Robotics*, *30*(11-12), 706–728.

Varela, F. J., Thompson, E., & Rosch, E. (2017). *The embodied mind: Cognitive science and human experience*. The MIT press.

Wilson, M. (2002). Six views of embodied cognition. *Psychonomic Bulletin & Review*, *9*(4), 625–636. https://doi.org/10.3758/BF03196322